gitlab-ci example
Elías Morales Valencia authored Mar 17, 2025

Project information
Ejemplo de microservicio, área de arquitectura.

Created on
December 27, 2023

Name	Last commit	Last update
.gitlab/merge_request_t...	mr template	1 year ago
gradle/wrapper	Inicia proyecto ejemplo	2 years ago
src	se arregla inyeccion de dep...	1 year ago
.gitlab-ci.yml	gitlab-ci example	1 year ago
Dockerfile	example.	1 year ago
README.md	Edit README.md	1 year ago
Zurich-Santander-Chile....	logo	1 year ago
build.gradle	Se agrego la conexion del a...	1 year ago
gradlew	Inicia proyecto ejemplo	2 years ago
gradlew.bat	Inicia proyecto ejemplo	2 years ago
settings.gradle	Inicia proyecto ejemplo	2 years ago

README.md

Example microservice

Spring Boot 3.4.0 | Spring Data JPA | Spring Validation | Oracle JDBC 21.9.0 | Lombok 1.18.36 | SpringDoc OpenAPI | Java 17

- **Get started with shield.io badges**

Table of Contents

- Description
- Authors
- Technologies Used
- API
- Prerequisites
- Configuration
- Run the application

Description

A comprehensive description of your application is required, outlining its functionality, purpose, and integration capabilities. The description shold be clear, concise, and accurrate.

Authors

Company logo
- This project was developed by ZS Arquitectura TI
- Elías Morales Valencia (IAC, cicd, publish app) - DevOps
- Elías Zarate Mancilla (Source code) - Integration Lead.

Technologies Used

- **Spring Boot:** Main framework for building the application.
- **Spring Data JPA:** Provides support for data access using JPA (Java Persistence API).
- **Spring Validation:** Utilized for validation of input data.
- **Oracle JDBC:** Java Database Connectivity for Oracle databases.

- **Lombok:** Library to reduce boilerplate code in Java.
- **SpringDoc OpenAPI:** Integration for API documentation using Swagger.

API

- **URL:** http://localhost:8080/base/path.
- **Port:** 8080 or port number when it has been deployed.
- **Documentation** URL DOCUMENTACION

Prerequisites

- **Java**: Ensure Java 11+ is installed.
- **Maven**: Install Maven to manage dependencies.
- **Oracle Database**: The service connects to an Oracle database instance.
- **Environment Variables**: Configure environment variables as described below.

Configuration

Environment variables

The application requires the following to be set in the system environment variables.

Variable Name	Description	Example Value
`AZURE_CLIENT_ID`	Azure client ID for authentication	`your-client-id`
`AZURE_CLIENT_SECRET`	Azure client secret	`your-secure-client-secret`
`AZURE_TENANT_ID`	Azure tenant ID	`your-tenant-id`
`APPCFG_LABEL`	Configuration label for the application	`example`
`URL_APP_CONFIGURATION`	Configuration URL for the application	`https://your-appcfg.azconfig.io`

Azure app configuration Variables

The application requires the following variables to be configured in Azure app configuration

Variable Name	Description	Example Value
`servicioejemplo.iaxis.urlOraAxis`	JDBC URL of the Oracle database	`jdbc:oracle:thin:@db-`
`servicioejemplo.iaxis.usernameOraAxis`	Oracle database username	`AXIS`
`servicioejemplo.iaxis.passwordOraAxis`	Oracle database password	`your-secure-password`
`servicioejemplo.iaxis.token`	oAuth2 get token URL	`https://uat-apigw.santanderseguros`
`servicioejemplo.iaxis.clientid`	oAuth2 client id	`your-client-id`
`servicioejemplo.iaxis.granttype`	oAuth2 granttype	`client credentials`

Variable Name	Description	Example Value
`servicioejemplo.iaxis.getUFURL`	oAuth2 get UF microservice url	`https://uat-apigw.san utils/getUf`

Application properties

src/main/resources/bootstrap.properties:

Param	Description	Value
`spring.cloud.azure.appconfiguration.stores[0].endpoint`	APPCFG URL Environment var	`${URL`

src/main/resources/application.properties:

Param	Description	Value
`spring.datasource.url`	appcfg variable with iaxis db url	`${servicioejemplo.urlOraAxis}`
`spring.datasource.username`	appcfg variable with iaxis db username	`${servicioejemplo.usernameOra`
`spring.datasource.password`	appcfg variable with iaxis db password	`${servicioejemplo.passwordOra`
`api.utilidades.token`	appcfg variable with get oauth2 token url	`${servicioejemplo.token}`
`api.utilidades.token.clientid`	appcfg variable with oauth2 token client id	`${servicioejemplo.clientid}`
`api.utilidades.token.granttype`	appcfg variable with oauth2 token granttype	`${servicioejemplo.granttype}`
`api.utilidades.uf`	appcfg variable with UF WS url	`${servicioejemplo.getUFURL}`

Run the application

Prerequisites

- **Docker Installed:** Ensure Docker is installed and running on your system. Refer to the [Docker installation guide] for instructions.

Steps to Run with Docker

1. **Build the Docker Image**

   - 📌 **Using** `Dockerfile`: It uses environment variables

     ```
     docker build -t myapp:v1.0.0 .
     ```

2. **Run the Docker Container**

   📌 Start application:

   ```
   docker run -d --name myapp \
   -e APPCFG_LABEL=example \
   -e AZURE_CLIENT_ID=client-id \
   -e AZURE_CLIENT_SECRET=client-secret \
   -e AZURE_TENANT_ID=tenant-id \
   -p 8080:8080 example:latest \
   myapp:v1.0.0
   ```